

## CS3070 Operating Systems

**Proof by Contradiction:** Assume  $k$  processes (where  $k \geq 2$ ) are currently in the critical section, protected by the atomic lock. All operations on the semaphore's internal state are protected by `self.lock.acquire(caller)` in both internal critical sections, ensuring they act as a single atomic critical section for the semaphore's internal state.

Since all  $k$  processes are in the critical section, each must have successfully executed `self.lock.acquire(caller)`.

**Contradiction:** Some process goes first and executes `self.lock.acquire(caller)`, acquiring the lock exclusively. When another process attempts `self.lock.acquire(caller)`, it cannot return because the lock is already held. However, our assumption states this other process (and all subsequent processes) are already in the critical section, which directly contradicts the fact that their `self.lock.acquire(caller)` calls have not yet returned.

Therefore, our assumption is false, and at most one process can be in the critical section at any time.

D2 – Discuss what insights have your group has gained from implementing a Semaphore.

We learned how mathematical proofs can verify the correctness of concurrent systems by establishing invariants and proving that our code maintains these invariants under all possible execution scenarios. This approach ensures data integrity and proper synchronization in multiprocessing environments.